Project Export

Domain: Cybersecurity Export: Single project Projects: 1 Generated: 9/13/2026, 11:42:13 AM

1. CTF-Builder - A Platform for Creating Capture The Flag Events

Domain: Cybersecurity Slug: ctf-builder-capture-the-flag-platform Created: 1/13/2026

Summary

CTF-Builder is a specialized SaaS platform enabling users to host their own cybersecurity competitions. This project involves building a real-time scoring engine using WebSockets, a challenge management CMS, and a robust flag validation system.

Full Content

Project Overview

Case Study: CTF-Builder

Capture The Flag (CTF) competitions are the standard for training and recruiting cybersecurity talent. However, setting up the infrastructure—provisioning servers, managing user accounts, and building a real-time scoreboard—is a logistical nightmare. Often, the organizers spend more time fixing the platform than designing the hacking challenges.

"CTF-Builder" democratizes this process. It is a SaaS platform where anyone (university professors, team leads, or hiring managers) can spin up a hosted CTF event in minutes.

As a Full Stack Security Engineer Intern, you will build a platform that serves two distinct masters: the Organizer (who needs a CMS to manage content) and the Hacker (who needs a responsive interface to solve challenges). The core technical challenge is State Management: managing the game state (scores, unlocks, first-blood achievements) in real-time for hundreds of concurrent users.

Key Objectives & Technical Architecture

Phase 1: Challenge Logic & Database

- Data Models: You need a robust relational schema (using PostgreSQL).
    - Event: Name, StartTime, EndTime.
    - Challenge: Title, Description, Category (Web, Crypto, Pwn), Points, The Flag (hashed or plaintext).
    - Submission: Linked to User/Team + Challenge. Must record the timestamp.
- Flag Validation: The core mechanic.
    - Logic: User inputs CTF{s3cr3t_fl4g} $

ightarrow$ Backend compares hash $ ightarrow$ If correct, award points.

    - Anti-Brute Force: Implement Rate Limiting (using Redis) to prevent users from scripting a dictionary attack against the flag submission endpoint.

Phase 2: The Organizer Dashboard (Admin)

- Event Wizard: A multi-step form to configure the event branding and duration.
- Challenge Editor: A CMS interface.
    - Features: Markdown support for descriptions. File uploads (providing the "pcap" or "binary" files for download). Dynamic flag generation.

Phase 3: The Real-Time Scoreboard

- The Architecture: In the final minutes of a CTF, the scoreboard is hit aggressively. Querying Postgres every time is too slow.
    - Solution: Use Redis Sorted Sets to maintain the leaderboard in memory.
- WebSockets: Integrate Socket.io.
    - Flow: User submits flag $

ightarrow$ Server validates $ ightarrow$ Server updates Redis $ ightarrow$ Server emits UPDATE_SCOREBOARD event $ ightarrow$ All connected clients update their UI instantly without refreshing.

Phase 4: Team Management & Access Control

- Team Mode: CTFs are often played in teams.
    - Logic: If Player A solves a challenge, Player B (on the same team) sees it as solved, and the team gets the points. This requires complex database joins.
- Access Control: Ensure that participants cannot access the Admin API routes to peek at the flags before the event starts.

Prerequisites

- Languages: JavaScript/TypeScript.
- Frontend: React.js.
- Backend: Node.js.
- Database: PostgreSQL (Core Data), Redis (Leaderboard/Caching).
- Real-Time: Socket.io/WebSockets.

Deliverables

1. The GitHub Repository

- Monorepo Structure: Separate folders for client-admin, client-participant, and server.
- Seed Script: A script that populates the database with a "Demo Event" containing 5 challenges (Web, Crypto, Forensics).

2. The Live Application

- Deployed and playable.
- Demo Accounts: Provide credentials for an "Admin" and a "Player" in the README.

3. Game Logic Documentation

- Explain your scoring formula (e.g., "We use dynamic scoring: the more people solve a challenge, the fewer points it is worth").

Project Timeline & Deadlines

- Week 1: Core Engine
  - Goal: The Rules.
  - Tasks: Schema Design. User/Team Auth. Challenge CRUD (Admin side). Flag Validation endpoint.
- Week 2: The Player Interface
  - Goal: Interaction.
  - Tasks: Build the Challenge List UI (filter by category). Build the Submission input. Handle "Correct/Incorrect" feedback states.
- Week 3: The Scoreboard (Real-Time)
  - Goal: Competition.
  - Tasks: Set up Redis. Implement the Socket.io server. Build the animated Scoreboard graph (e.g., lines showing rank over time).
- Week 4: Polish & Deployment
  - Goal: Launch.
  - Tasks: Implement Team Logic (shared solves). Add File Uploads for challenge attachments. Final Deployment and Load Testing.